Blind Trust — Guida operativa step-by-step per i partecipanti

> **Missione:** agire come Security Enablement Team di AstraShop. Prima del go-live dovete trovare, dimostrare in modo innocuo, classificare e mitigare rischi introdotti da codice o configurazioni accettati con fiducia eccessiva in suggerimenti IA.**Regola guida:** raccogliete prove, non supposizioni. Il valore della consegna non è il payload più creativo, ma la qualità dell'evidenza, della patch proposta, del test e della prevenzione.

---

1. Prima di iniziare: organizzate il team

1.1 Formate un team da 2–4 persone

Assegnate ruoli leggeri; potete alternarli a ogni flag.

Ruolo	Cosa fa
Navigator	Guida l'analisi dell'app e del repository; condivide lo schermo.
Code reviewer	Legge codice, configurazioni e history Git; annota file e righe rilevanti.
Evidence keeper	Compila la scheda finding mentre il team lavora.
Patch/test owner	Trasforma la remediation in una patch o in pseudocodice e definisce il test di regressione.

1.2 Leggete le regole di ingaggio

Prima di aprire l'app, confermate tutti questi punti:

1. Operate **solo** nella cartella e nell'istanza AstraShop assegnata.
2. Usate **solo** utenti demo, dati seed e token con prefisso `TRAINING_ONLY_`.
3. Non usate domini, registry, account, repository o scanner esterni al laboratorio.
4. Non inserite segreti, prompt, configurazioni o dati reali nell'app, nel terminale o nelle note.

5. Fermatevi alla **PoC minima e innocua**; dopo aver confermato il rischio, documentate e passate alla mitigazione.
6. Non aprite `docs/safe-patch-guide.md` o `training/expected-findings.md` prima del debrief, salvo indicazione del facilitatore.

---

2. Avviate e verificate il laboratorio

2.1 Aprite il progetto

Dopo aver estratto lo ZIP o clonato il repository:

```bash
cd astrashop
node --version
```

Serve Node.js **22 o superiore**.

2.2 Avviate l'app senza installare nuove dipendenze

Il laboratorio non richiede moduli runtime esterni. Per mantenere intatta la challenge supply chain, **non eseguite `npm install` autonomamente all'inizio**: l'installazione di una dipendenza è essa stessa parte dell'analisi della Flag 1.

Avviate invece direttamente:

```bash
npm start
```

Aprite nel browser:

```
http://localhost:3000
```

> Se avete già eseguito `npm install`, non avete contattato servizi esterni: l'eventuale effetto previsto è una sola riga nel log locale `data/telemetry.log`. Annotatelo e, se necessario, usate il reset del lab prima di iniziare la Flag 1.

2.3 Fate il controllo iniziale

1. Nella dashboard verificate di vedere cinque flag: `FLAG_1` – `FLAG_5`.

2. Nel selettore **Identità demo** controllate che compaiano Alice, Bruno e Ops Team.
3. Aprite **Regole & guide** nell'app e rileggete i confini.
4. Aprite nel repository `README.md` e `training/finding-template.md`.
5. Create un file di consegna per il team, ad esempio `consegne/team-<nome>-findings.md`, copiando il template.

Se l'app non si avvia, avvisate il facilitatore prima di modificare configurazioni o scaricare strumenti.

---

3. Usate lo stesso metodo per ogni flag

Per ogni challenge seguite questa sequenza. Non saltate direttamente alla patch.

1. **Osservate.** Individuate il comportamento o l'artefatto sospetto nella UI, nel repository o nella history.
2. **Formulate un'ipotesi.** Scrivete quale assunzione non verificata potrebbe essere errata.
3. **Tracciate il flusso.** Cercate origine del dato, trasformazioni, controllo, uso finale e confine di autorizzazione.
4. **Riproducete in sicurezza.** Fate la prova più piccola possibile, esclusivamente con record demo e nel lab.
5. **Raccogliete evidenza.** Salvate file, funzione/route, passaggi, risultato osservato e impatto potenziale.
6. **Proponete remediation.** La proposta deve includere modifica tecnica, test di regressione e controllo preventivo di processo/CI.
7. **Classificate.** Indicate severità, confidenza, owner e priorità.

Per ogni flag compilate nella scheda:

- assunzione errata;
- evidenza nel lab;
- impatto potenziale nel mondo reale;
- patch proposta;
- test di regressione;
- controllo preventivo.

---

4. FLAG_1 — Il pacchetto che suona familiare

Obiettivo: dimostrare perché una dipendenza suggerita da IA non va adottata senza verificarne provenienza e governance.

Passi

1. Nel repository aprite `package.json` e annotate ogni dipendenza non standard.
2. Cercate riferimenti al package sospetto in:

   ```bash
   rg -n "text-normalizer|suggestedPackage|postinstall" package.json packages/
   ```

3. Leggete, in quest'ordine:
   - `docs/llm-development-notes.md`;
   - `training/registry-metadata.json`;
   - `packages/acme-text-normalizer/package.json`;
   - `packages/acme-text-normalizer/postinstall.mjs`.
4. Rispondete: il package è in un catalogo approvato? Ha owner, repository sorgente, manutenzione, licenza, integrity e processo di review verificabili?
5. Individuate **almeno tre segnali** che devono bloccarne l'adozione. Esempi di categorie: package name solo plausibile, owner generico, repository assente, pubblicazione recente, metadati incoerenti, script lifecycle, assenza di approvazione.
6. Solo dopo l'analisi, usate il pulsante **Simula installazione locale** nel Repository explorer oppure eseguite:

   ```bash
   npm run simulate:package
   ```

7. Controllate il solo log locale:

   ```bash
   cat data/telemetry.log
   ```

8. Nella consegna, proponete una delle due strade:
   - rimozione della dipendenza e uso di una funzione standard/sicura; oppure
   - adozione solo dopo verifica formale, lockfile, scope/registry approvata e review dedicata.

9. Definite il gate CI: ad esempio allowlist delle registry, review obbligatoria per nuove dipendenze, SCA e SBOM di release.

Evidenza minima: nome della dipendenza, tre segnali di rischio, comportamento osservato solo nel log locale, patch/policy proposta.

---

5. FLAG_2 — Sanitized ≠ Safe

Obiettivo: dimostrare che il nome di una funzione non è una garanzia; conta il flusso reale da input a sink.

Passi

1. Nell'app selezionate **demo-alice**.
2. Aprite **Ticket & report** e selezionate `TKT-1024`.
3. Prima di inserire qualunque dato, leggete:
   - `src/middleware/sanitize.mjs`;
   - `src/services/report-renderer.mjs`;
   - la parte relativa alla preview in `public/app.js`.
4. Disegnate rapidamente il flusso:

   Plain Text

   ```
   textarea → POST /api/tickets/:id/note → sanitizeNote( ) → ticket.notes
   → GET /api/reports/:id?preview=true → noteHtml → preview report
   ```

5. Inserite **solo** questo tipo di prova innocua nella nota, sostituendo il nome del vostro team:

   HTML

   ```
   <strong>MARKER TEAM ALFA</strong>
   ```

6. Salvate la nota e selezionate **Apri preview report**.
7. Registrate il risultato visibile: il marcatore viene interpretato come HTML anziché essere mostrato come testo.
8. Non provate script, event handler, URL esterni, esfiltrazione, accesso a cookie o payload ulteriori. La challenge termina con il marcatore visibile.

9. Proponete la patch: rendere la nota come testo (`textContent` o auto-escaping contestuale), eliminare il sink HTML non necessario e mantenere una validazione coerente con il tipo di dato.
10. Definite un test di regressione: una nota con `<strong>MARKER</strong>` deve comparire letteralmente nel report, non come markup interpretato.

Evidenza minima: diagramma source → trasformazioni → sink, prova innocua, spiegazione del controllo insufficiente, patch e test.

---

6. FLAG_3 — Il segreto nel contesto

Obiettivo: trattare un segreto esposto come compromesso nel mondo reale, anche se nel lab è una stringa esca.

Passi

1. Cercate indicatori locali senza contattare servizi esterni:

```bash
rg -n "TRAINING_ONLY|API_KEY|Bearer|token|secret" --glob '!node_modules' .
```

2. Ispezionate in particolare:
   - `docs/llm-development-notes.md` ;
   - `data/debug.log` ;
   - `.env.example` ;
   - eventuali fixture e file di configurazione.
3. Cercate una traccia nella cronologia Git:

```bash
git log --all --oneline
git log -p --all -- legacy/.env
```

4. Non tentate di usare nessun valore trovato. È un'esca fittizia; nel mondo reale sarebbe comunque da considerare compromessa fino a prova contraria.
5. Compilate un mini incident ticket con queste cinque sezioni:
   - **perimetro:** dove compare il valore (file attivo, log, documentazione, history);
   - **azione immediata:** revoca/rotazione presso il provider, se fosse reale;

- **contenimento:** rimozione da file, log, issue, artifact e prompt;
- **eradicazione:** pulizia della history Git con una procedura approvata;
- **prevenzione:** secret manager, secret scanning pre-commit e CI, minimizzazione del contesto inviato a LLM.

6. Proponete una regola di team: quali informazioni non possono entrare in prompt, ticket, screenshot o snippet usati per assistenza IA.

Evidenza minima: artefatti individuati, history controllata, piano di risposta all'incidente e prevenzione.

---

7. FLAG_4 — Autenticato non significa autorizzato

Obiettivo: distinguere "chi è l'utente?" da "può leggere proprio questa risorsa?".

Passi

1. Nella UI passate da **demo-alice** a **demo-bruno**.
2. Aprite **Bozze d'ordine**.
3. Verificate gli ordini normalmente visibili a Bruno.
4. Nella casella **ID ordine** inserite `ORD-1001`, che appartiene ad Alice, e selezionate **Carica draft**.
5. Registrate l'evidenza: l'app dichiara l'identità richiedente e l'owner effettivo della bozza.
6. Leggete la route corrispondente in `server.mjs`, cercando `GET /api/orders/:id/draft` oppure:

```bash
rg -n "orders/.+draft|findOrder|ownership|tenant" server.mjs
```

7. Spiegate il difetto in una frase: la route verifica un'identità demo valida, ma ricerca l'ordine per ID senza vincolare owner e tenant al chiamante.
8. Proponete la correzione server-side, idealmente nel punto di query/service:

```
const order = state.orders.find((candidate) =>
  candidate.id === orderId &&
  candidate.ownerId === user.id &&
  candidate.tenantId === user.tenantId
);
```

In un'app reale, la policy deve considerare ruoli autorizzati espliciti e non basarsi esclusivamente sul client.

9. Definite due test: un caso consentito (Alice → `ORD-1001`) e un caso negato (Bruno → `ORD-1001`).

Evidenza minima: riproduzione con sole identità demo, route/query coinvolta, test negativo e patch proposta.

---

8. FLAG_5 — La pipeline non può fidarsi del codice

Obiettivo: trasformare i finding precedenti in controlli ripetibili prima del merge.

Passi

1. Aprite `.github/workflows/ci.yml`.
2. Elencate cosa la pipeline fa già: sintassi e test baseline.
3. Elencate cosa **non** fa, collegandolo alle flag appena trovate:

Rischio	Controllo da proporre	Quando	Azione suggerita
Segreti in repo/log/history	Secret scanning	pre-commit, PR, repository	blocco per secret verificabile
Pattern vulnerabili nel codice	SAST	ogni PR	blocco su criticità/alta confidenza concordata
Dipendenza non approvata o vulnerabile	SCA e policy registry/lockfile	ogni PR	blocco se fuori policy
Inventario componenti	SBOM	build di release	artifact obbligatorio
Regressione di autorizzazione/input	test unitari e di sicurezza	ogni PR	blocco

1. Per ogni controllo decidete: cosa blocca, cosa genera warning e chi può approvare un'eccezione.
2. Definite come gestire falsi positivi: owner, motivazione, data di scadenza e rivalutazione.
3. Aggiungete almeno una misura non automatizzabile: review AppSec, checklist PR per output LLM, formazione o ownership delle dipendenze.
4. Consegnate una delle due opzioni richieste dal facilitatore:

- patch della pipeline; oppure
- disegno tecnico con tool, comando, trigger, soglia e comportamento in caso di fallimento.

Evidenza minima: almeno un controllo per segreti, codice e dipendenze, con gate e gestione eccezioni motivati.

---

9. Consegna finale del team

Al termine, consegnate **cinque schede finding** (una per flag) oppure una singola relazione con cinque sezioni. Ogni finding deve avere:

Sezione	Domanda a cui risponde
Titolo e severità	Che cosa è stato trovato e quanto è urgente?
Assunzione errata	Quale fiducia non verificata ha permesso il rischio?
Evidenza	In quale file/flusso e con quale prova innocua?
Impatto	Che cosa potrebbe accadere in un sistema reale?
Patch	Quale modifica risolve la causa, non solo il sintomo?
Test	Come impedire la regressione?
Prevenzione	Quale controllo di processo o CI riduce la probabilità di ricaduta?
Owner/priorità	Chi agisce e con quale scadenza?

Chiudete con una raccomandazione **go / no-go condizionata** per AstraShop, ad esempio:

> **No-go fino alla correzione delle flag ad alta priorità**; il rilascio può riprendere dopo rimozione della dipendenza non verificata, correzione del rendering HTML e dell'authorization, gestione dell'esposizione dei segreti e introduzione dei gate CI concordati.

---

10. Reset e chiusura

1. Salvate la vostra consegna fuori dalla cartella che verrà resettata, se il facilitatore lo richiede.
2. Nella UI aprite **Finding board** e usate **Reset completo** per ripristinare note, finding e telemetria del vostro lab.
3. Fermate il server con `Ctrl+C` nel terminale in cui è in esecuzione `npm start`.
4. Se state usando Docker, eseguite:

   ```bash
   docker compose down
   ```

5. Non copiate nel materiale di debrief alcuna configurazione o dato non appartenente all'ambiente fittizio.
6. Partecipate al debrief portando una lezione tecnica e una modifica di processo che il vostro team potrebbe adottare davvero.

> **Messaggio conclusivo:** l'IA accelera la produzione di ipotesi e codice; non sostituisce l'evidenza, la review umana, i test e i controlli di supply chain.